



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

One Statement, Three Meanings

CMSC 124 Activity 6

Prepared by: Rene Andre Bedonia Jocsing

Published: September 14, 2026

Deadline: September 14, 2026

Activity 3 turned characters into tokens. Activity 4 arranged those tokens into syntax trees, and Activity 5 checked static facts such as type compatibility. This activity assumes that an assignment passed those checks. The next question is what the assignment does when it runs.

Lesson 2 described an imperative program as a sequence of memory changes. **Dynamic semantics** gives those changes a precise meaning. Today you'll execute one two-line program, model it as a mathematical function, and prove what result it establishes.

The programs here are pseudocode. They're written to be read. Every rule they depend on is on this page.

All variables in this activity hold integers.

You have 60 minutes for the four checkpoints. Keep each worked example beside the checkpoint it supports. Discuss with classmates and the instructor at any point. This activity is worth 20 points. Twelve points reward traces or derivations, and eight reward their explanations.

State and the Running Program

A **state** maps each variable to its current value. This is Lesson 2's memory-cell model written as a mathematical mapping. We write $\{x \rightarrow 3, y \rightarrow 0\}$ for a state where x holds 3 and y holds 0.

Read $\{x \rightarrow 3, y \rightarrow 0\}$ as "the state maps x to three and y to zero."

Start with that state and run:

```
1 x = x + 4;  
2 y = 2 * x;
```

An assignment evaluates its right side in the old state, then replaces only the target's mapping. Every other mapping stays unchanged. A sequence finishes the first statement before starting the second.

Notation You'll Meet

Symbol	Meaning
s	one program state
$S, S1, S2$	statements
E	an arithmetic expression on an assignment's right side
P, Q	conditions before and after a statement
B	a loop guard
I	a proposed loop invariant

The symbol $=$ has two jobs in this pseudocode. Inside $x = x + 4$, it assigns a new value to x . Inside a condition such as $x = 7$, it states that two values are equal. The surrounding notation tells you which job it has.

Operational Meaning

Operational semantics explains a program through execution steps on an abstract machine. For an assignment, show the right-side lookup, the computed value, and the updated state. These are separate steps. Writing only the final state skips the steps we care about.

Worked Example for Checkpoint 1

Suggested time: 8 minutes, including the worked example.

Start from $\{z \rightarrow 3\}$ and execute $z = z + 2$.

Step	Result
look up z	3
compute $3 + 2$	5
update only z	$\{z \rightarrow 5\}$

Checkpoint 1: Execute It (4 Points)

Begin with state $\{x \rightarrow 3, y \rightarrow 0\}$ and execute these assignments in order:

```

1  x = x + 4;
2  y = 2 * x;
```

1. (Construction: 2 points) Trace both assignments from the initial state. Show every lookup, arithmetic result, and state after an assignment.

2. (Explanation: 2 points) Explain why the second statement must read the new value of x . Cite the sequence rule that excludes the old value.

Denotational Meaning

The operational trace for $z = z + 2$ listed a lookup, an addition, and an update. We can also describe the same meaning using only its input and output:

```

1  input {z -> 3} -> z = z + 2 -> output {z -> 5}
```

Denotational semantics maps a program phrase to a mathematical object. For these statements, that object is a function from one state to another. Let $M[S](s)$ mean the state produced by statement S from input state s .

Read $M[S](s)$ as “the meaning of statement S applied to state s .”

In the assignment rule below, E is the right-side arithmetic expression. To find its value in s , look up every variable in E using s , then perform the arithmetic. The x in the general rule stands for whichever variable the assignment targets.

```
1  M[x = E](s) = s with x replaced by the value of E in s
```

For a sequence, function composition preserves program order:

```
1  M[S1; S2](s) = M[S2](M[S1](s))
```

Read the right side from the inside out: “apply $S1$ to s , then apply $S2$ to the resulting state.”

Worked Example for Checkpoint 2

Suggested time: 8 minutes, including the worked example.

Start from $\{a \rightarrow 2, b \rightarrow 1\}$ and use this fresh sequence:

```
1  a = a + 3;
2  b = 4 * a;
```

Apply the sequence rule from the inside out. The first function reads $a = 2$, computes 5, and returns a state with only a replaced:

```
1  M[a = a + 3]({a → 2, b → 1})
2  = {a → 5, b → 1}
```

That whole mapping becomes the input to the second function:

```
1  M[b = 4 * a](M[a = a + 3]({a → 2, b → 1}))
2  = M[b = 4 * a]({a → 5, b → 1})
3  = {a → 5, b → 20}
```

The second update changes b and leaves a untouched. The function description shows the intermediate and final states without listing machine steps.

Checkpoint 2: Build the Function (4 Points)

Use these rules with input state $\{x \rightarrow 3, y \rightarrow 0\}$:

```
1  M[x = E](s) = s with x replaced by the value of E in s
2  M[S1; S2](s) = M[S2](M[S1](s))
```

```
1  x = x + 4;
2  y = 2 * x;
```

1. (Construction: 2 points) Apply the two semantic functions to the running state. Write the intermediate mapping passed from the first function to the second and the final mapping returned by the composition.

2. (Explanation: 2 points) Explain how this function description states the same meaning as your operational trace even though it doesn't mention machine steps. Consider what the trace shows that the function hides, and what the function shows plainly that the trace buries.

Axiomatic Meaning

Execution starts with an old state. Suppose we know the result we want. Which conditions on that old state guarantee the result?

Axiomatic semantics answers that question with conditions and proof rules. A **precondition** P describes what must be true before statement S . A **postcondition** Q describes what must be true afterward. A **Hoare triple** $\{P\} S \{Q\}$ says that if P is true before S and S terminates, then Q is true afterward.

Read $\{P\} S \{Q\}$ as "if P holds before S , then Q holds after S , provided S terminates."

The **weakest precondition**, written $wp(S, Q)$, is the least restrictive condition that guarantees Q . A weaker condition permits more initial states. For example, $x = 3$ permits any initial value of y , while $x = 3$ and $y = 0$ permits only one.

Read $wp(S, Q)$ as "the weakest precondition for statement S and postcondition Q ."

Work backward with these rules:

```
1 wp(x = E, Q) = Q with E substituted for x
2 wp(S1; S2, Q) = wp(S1, wp(S2, Q))
```

Substitution replaces each occurrence of the assigned variable in the condition with the assignment's right side. It doesn't change the assignment statement. Simplify one step at a time.

Both jobs of $=$ now sit on the same line. In $wp(z = z + 2, z = 5)$, the first $=$ assigns and the second states an equality. Position is what tells them apart.

Worked Example for Checkpoint 3

Suggested time: 14 minutes, including the worked example.

For $z = z + 2$ with desired postcondition $z = 5$, substitute the right side $z + 2$ for z in the postcondition:

```
1 wp(z = z + 2, z = 5)
2 = z + 2 = 5
3 = z = 3
```

The Hoare triple is $\{z = 3\} z = z + 2 \{z = 5\}$.

The substitution runs one way only. This mistaken calculation puts the desired value into the statement's right side:

```
1 wp(z = z + 2, z = 5)
2 = wp(z = 5 + 2, z = 5)    <- 5 replaced z only on the right side
3 = 7 = 5
```

That calculation changes the statement and then gets a false condition. The rule must leave the statement alone. It replaces z inside the postcondition with $z + 2$, which produces a condition on the old state.

Checkpoint 3: Work Backward (6 Points)

Work with this sequence and required postcondition $y = 14$:

```
1  x = x + 4;
2  y = 2 * x;
```

1. (Construction: 4 points) Compute the weakest precondition before $y = 2 * x$, then the weakest precondition before $x = x + 4$. Show each substitution and algebraic simplification.

2. (Explanation: 2 points) Explain why working forward from the given state can confirm one run but can't establish the weakest condition for every permitted initial state. A forward trace starts from one state. A weakest precondition has to cover every state that could reach the postcondition.

Loops Need an Invariant

A finite sequence lets us apply wp backward one statement at a time. A loop may execute an unknown number of times, so we need one condition that connects all its iterations. A **guard** is the true-or-false condition checked before each possible body execution. That moment is a **loop boundary**.

A **loop invariant** I is a condition that holds at every loop boundary. A proposed invariant must answer these questions:

1. Why is I true before the first guard check? This is **initialization**.
2. If I and guard B are true, why does one complete body execution restore I ? This is **preservation**.
3. If I is true and B is false, why does postcondition Q follow? This is the **exit implication**.

These obligations prove **partial correctness**. They show that the postcondition holds if the loop terminates. A separate termination proof turns partial correctness into **total correctness**.

Worked Example for Checkpoint 4

Suggested time: 30 minutes, including the worked example.

Trace this loop:

```
1  i = 0;
2  total = 0;
3  // First boundary: check the guard here.
4  while (i < 3) {
5      i = i + 1;
6      // Temporary state: the body is not finished.
7      total = total + 2;
8      // Restored boundary: check the guard again.
9  }
```

The four figures below trace the moments before each guard check. The states stay in place. The blue marker moves through them as the loop runs.

Trace the loop at guard boundaries

Blue marks the current guard check. Green marks completed checks, and gray marks checks the loop hasn't reached yet.

Check the guard at entry

The initial assignments create $(0, 0)$. The guard $i < 3$ is true, so the body runs.

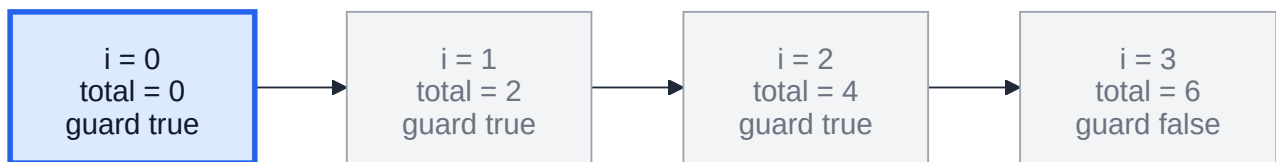


Figure 1: Check the guard at entry

Return after one iteration

One complete body execution changes (0, 0) to (1, 2). The guard remains true.

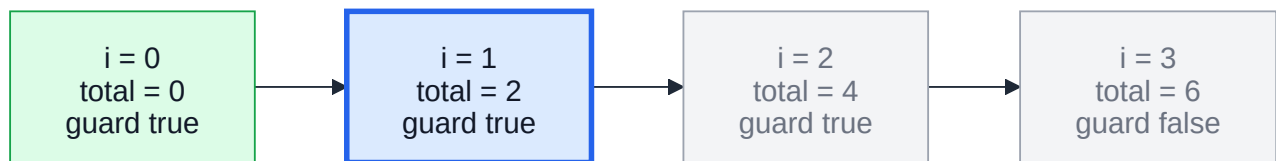


Figure 2: Return after one iteration

Return after two iterations

The second complete body execution reaches (2, 4). The guard remains true.

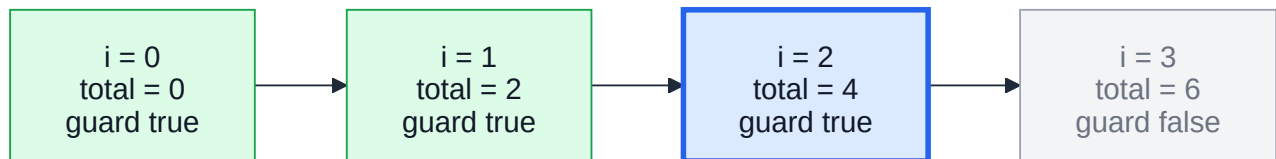


Figure 3: Return after two iterations

Reach the exit boundary

The third complete body execution reaches (3, 6). The guard is false, so the body stops.

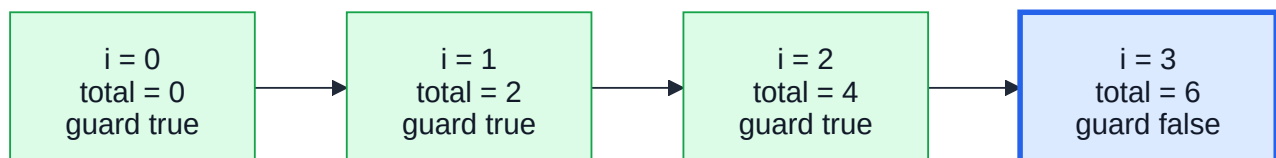


Figure 4: Reach the exit boundary

Find a Candidate Invariant

Use the boundary states (0, 0), (1, 2), (2, 4), and (3, 6) to test possible conditions. A useful invariant must survive every boundary and still help prove the postcondition at exit.

Candidate	Boundary check	What happens at exit?
<code>total = 6</code>	It fails at entry (0, 0).	It states the result, but failing at entry disqualifies it as this loop's invariant.
<code>total >= 0</code>	It holds at all four boundaries.	It permits values other than 6, so it is too weak.
<code>total = 2 * i</code>	It holds at all four boundaries.	With $i \leq 3$ and a false guard, it forces <code>total = 6</code> .

The guard allows the body to run only while $i < 3$. The body raises i by one, so the boundary values stay between 0 and 3. Add those bounds to the equality:

```
1 I: total = 2 * i and 0 <= i <= 3
```

Read I as “the invariant.” Read $0 \leq i \leq 3$ as “zero is less than or equal to i , and i is less than or equal to three.”

A candidate stays a guess until it clears initialization, preservation, and exit implication. For this fixed loop, the trace lists every reachable boundary. The figures below check entry, follow every body execution, and finish at the only exit boundary. The equality may be false between the two body statements. It must return after the complete body.

Verify the invariant after finding it

Concrete states check entry, every reachable body execution, and the exit boundary.

Establish the invariant at entry

At entry, $0 = 2 * 0$ and $0 \leq 0 \leq 3$, so I is true.

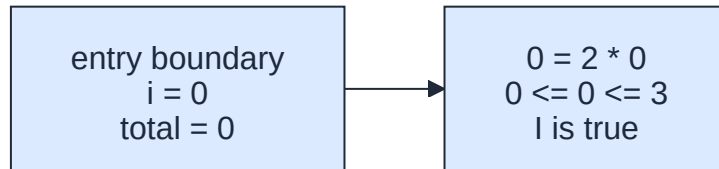


Figure 5: Establish the invariant at entry

Restore the invariant after every body execution

Follow every assignment from the entry boundary to the exit boundary. Each `total` assignment restores I .

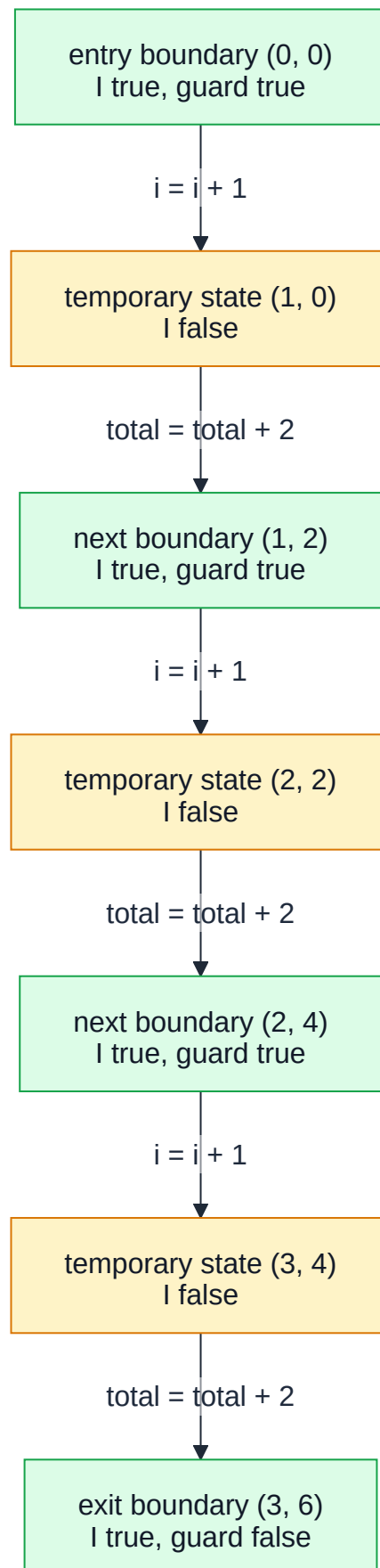


Figure 6: Restore the invariant after every body execution

Use the invariant at exit

At (3, 6), the invariant is true and the guard is false. Substitution gives the postcondition $\text{total} = 6$.

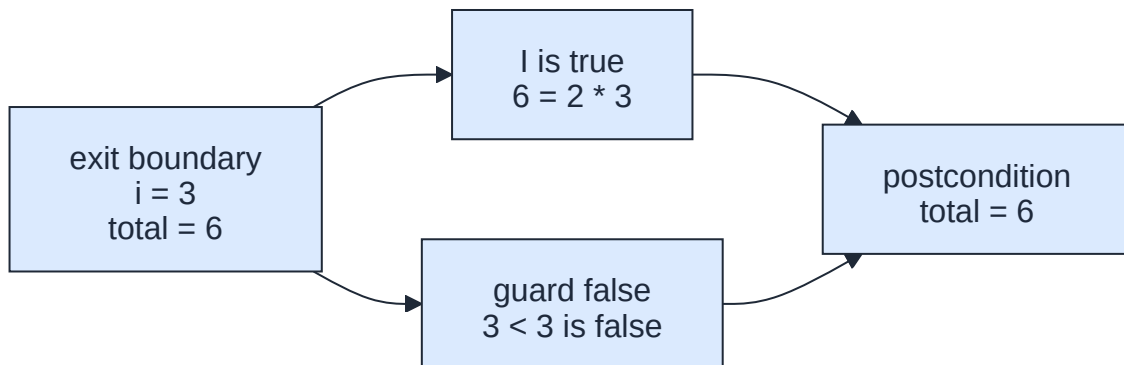


Figure 7: Use the invariant at exit

These concrete checks cover every reachable boundary in this loop. Checkpoint 4 uses the same three obligations, then asks you to write preservation with symbols so one step covers every boundary that satisfies the invariant.

Show That the Loop Stops

The invariant proof still has a condition. It establishes $\text{total} = 6$ if the loop stops. A **variant** is a nonnegative integer expression that decreases after every complete iteration while the guard is true. It supplies the separate proof that the loop must stop.

The guard $i < 3$ first becomes false at $i = 3$. The body raises i by one, so $3 - i$ counts the increments that remain. While the guard is true, this count is positive. Each complete iteration lowers it by one. A nonnegative integer can't decrease forever, so the loop must reach a false guard.

Show that the loop terminates

The guard supplies the stopping boundary. Subtracting the current counter gives the remaining increments.

Derive the variant from the guard

The guard $i < 3$ first becomes false at $i = 3$, so $3 - i$ counts how many increments remain.

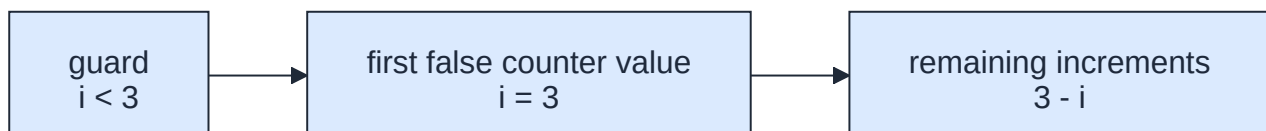


Figure 8: Derive the variant from the guard

Watch the variant fall

Each complete iteration raises i by one and lowers $3 - i$ by one until the guard becomes false.

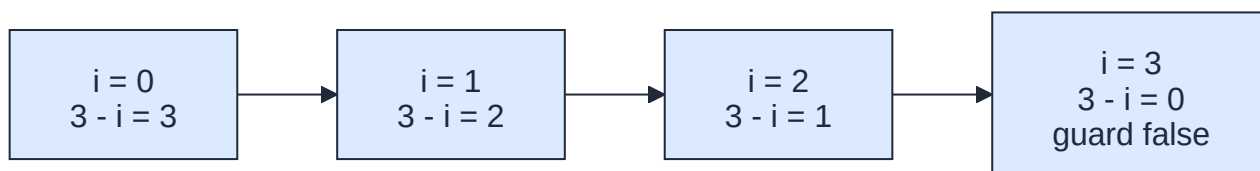


Figure 9: Watch the variant fall

Checkpoint 4: Prove the Loop (6 Points)**Transfer the Proof**

Carry the running program's final state $\{x \rightarrow 7, y \rightarrow 14\}$ into this loop:

```

1 while (x < 10) {
2   x = x + 1;
3   y = 2 * x;
4 }

```

The body adds one to x here, where the running program added four. That difference is deliberate. Adding four would carry x from 7 straight past 10 to 11, which breaks the bound $x \leq 10$ the invariant depends on.

The guard $x < 10$ first becomes false at $x = 10$, so $10 - x$ counts the remaining increments.

Use invariant $I: y = 2 * x$ and $x \leq 10$, postcondition $y = 20$, and variant $10 - x$. Complete the boundary table before writing the proof:

Boundary	Guard state (x, y)	Guard result	After $x = x + 1$	Restored state after $y = 2 * x$
entry				
after the 1st iteration				
after the 2nd iteration				
exit			not executed	not executed

1. (Construction: 4 points) Work in this order: fill the boundary table, establish the invariant at entry, preserve it through one symbolic body execution, and combine it with the false guard at exit. End with postcondition $y = 20$.

2. (Explanation: 2 points) Explain why $y = 2 * x$ needs the bound $x \leq 10$ to prove the exact postcondition. Then list the values of $10 - x$ at successive guard checks and explain how they prove termination separately from the three invariant obligations.